TOBB ETÜ
Ekonomi ve Teknoloji Üniversitesi

TOBB EKONOMİ VE TEKNOLOJİ ÜNİVERSİTESİ

Bilgisayar / Yapay Zekâ Mühendisliği

BİL 495-YAP 495

Yüksek Seviye Tasarım Raporu

Proje Adı	Akıllı Trafik Akış Simülasyonu ve Yol Tasarım Karar Destek Sistemi
Takım Üyeleri	Beril Aydın 231101002 Kaan Baykal 221101028 Duygu Akman 221101035 Yankı Yağız Ozan 231101059
Danışman	
Akademik Dönem	2025-2026 Yaz Dönemi
Teslim Tarihi	06.08.2026

İÇİNDEKİLER

1. GİRİŞ 3
1.1. Sistemin Amacı 3
1.2. Tasarım Hedefleri 4
1.2.1. Hedeflerin Tanımı 4
1.2.2. Çatışan Tasarım Hedefleri ve Tercih Gerekçeleri 5
1.3. Tanımlar, Kısaltmalar ve Kavramlar 6
2. MEVCUT YAZILIM MİMARİSİ 8
2.1. Mevcut Çalışma Biçimleri 8
2.2. Devralınan Mimari Unsur: SUMO'nun Dosya Tabanlı Süreç Modeli 8
3. ÖNERİLEN YAZILIM MİMARİSİ 9
3.1. Genel Bakış 9
3.2. Alt Sistem Ayrışımı 10
3.2.1. Alt Sistemlerin Sorumlulukları 11
3.2.2. Alt Sistem - Katman eşlemesi 12
3.2.3. Alt Sistemler Arası Arayüzler 12
3.2.4. Alt Sistemler Arası Veri Akışı 13
3.3. Donanım / Yazılım Eşlemesi 14
3.3.1. Dağıtım Birimleri 15
3.3.2. Dağıtım Senaryoları ve Taşınabilirlik 16
3.4. Kalıcı Veri Yönetimi 16
3.4.1. Veri Profilinin Analizi 16
3.4.2. Depolama Teknolojisi Seçimi 17
3.4.3. Kavramsal Veri Şeması 17
3.4.4. Dizin Düzeni ve Veri Yaşam Döngüsü 18
3.4.5. Yedekleme ve Kurtarma Stratejisi 19
3.5. Erişim Denetimi ve Güvenlik 20
3.5.1. Tehdit Yüzeyinin Belirlenmesi 20
3.5.2. Kimlik Doğrulama ve Yetkilendirme 20
3.5.3. Girdi Doğrulama ve Alt Süreç İzolasyonu 21
3.5.4. KVKK ve GDPR Uyumu 21
3.5.5. Şeffaflık ve Model Sınırlılığının Bildirimi 22
3.6. Küresel Yazılım Denetimi 22
3.6.1. Denetim Akışı Modeli 22
3.6.2. Eşzamanlılık Modeli 23
3.6.3. Hata Kurtarma Stratejisi 23
3.7. Sınır Koşulları 24
3.7.1. Soğuk Başlangıç (Cold Start) 24
3.7.2. Kapatma (Graceful Shutdown) 25

3.7.3. Hata ve İstisna Durumları..... 25
4. ALT SİSTEM SERVİSLERİ..... 26
5. HLD - LLD İZLENEBİLİRLİĞİ..... 28
6. SÖZLÜK..... 30
7. KAYNAKLAR..... 32
8. ARAÇLAR VE METODOLOJİ..... 33

1. GİRİŞ

1.1. Sistemin Amacı

Kentsel yol ağlarında yapılan geometrik ve operasyonel müdahalelerin (şerit kapatma, tek yöne çevirme, katılım şeridi ekleme vb.) etkileri, günümüzde çoğunlukla sahada deneme-yanılma yoluyla gözlemlenmektedir. Bu yaklaşım yüksek maliyetlidir, uygulama sürecinde trafiği aksatır ve hatalı bir kararın sonucu ancak yatırım tamamlandıktan sonra görülebilir. Geliştirilen sistem, bu kararların fiziksel uygulamaya geçilmeden önce sanal ortamda değerlendirilmesini sağlayan, web tabanlı ve açık kaynaklı bir "Ne Olurdu?" karar destek platformudur.

Sistemin işlevsel özü şudur: kullanıcı, hazır olarak yüklenmiş bir pilot bölge haritası üzerinde kod yazmadan topolojik müdahaleler tanımlar; sistem bu müdahaleleri ana haritayı bozmadan izole bir çalışma kopyasına uygular, ağın geçerliliğini doğrular, mikroskobik trafik simülasyonunu arka planda asenkron olarak koşturur ve sonuçları ısı haritaları, karşılaştırmalı metrik tabloları ve kural tabanlı darboğaz uyarıları biçiminde sunar. Simülasyon çekirdeği olarak açık kaynaklı SUMO (Simulation of Urban MObility) motoru [1] bir "kara kutu" biçiminde kullanılır; sistemin kendi mühendislik katkısı, bu motorun çevresinde kurulan veri hazırlama, senaryo yönetimi, doğrulama, orkestrasyon ve analiz katmanlarıdır.

Bu doküman, önceki fazlarda üretilen analiz modelini (gereksinimler, kullanım durumları, alan modeli, dinamik modeller) bir sistem tasarım modeline dönüştürmeyi amaçlamaktadır. Odak noktası "ne yapılacak" sorusundan "hangi mimari ile yapılacak" sorusuna kaymaktadır. Bu kapsamda alt sistemler ve sorumlulukları, alt sistemler arasındaki veri akışı, donanım/yazılım eşlemesi, kalıcı veri yönetimi, erişim denetimi ve güvenlik yaklaşımı, küresel denetim akışı ve sınır koşulları tanımlanmaktadır. Dokümanda sınıf, fonksiyon ve değişken düzeyindeki ayrıntılardan bahsedilmemiştir; bu ayrıntılar Düşük Seviye Tasarım (LLD) dokümanının konusudur. Bölüm 5'te sunulan izlenebilirlik matrisi, bu dokümandaki her mimari bloğun LLD'de hangi bileşen grubunda açılacağını göstermektedir.

Sistemin doğrulaması, New York City, Lower East Side pilot bölgesi üzerinde, yaklaşık 40-50 yol segmentlik bir ağ ölçeğinde yapılacaktır. Bu ölçek, standart donanımlı bir kişisel bilgisayarda kabul edilebilir sürede koşum yapılabilmesi kısıtından türetilmiştir. Doğrulama yöntemi olarak Bölüm 1.2.1'de TH6 kapsamında tanımlanan GEH istatistiği ve KL Iraksama metrikleri kullanılacak; simülasyon çıktıları gerçek sensör ölçümleri ve taksi seyahat kayıtlarıyla karşılaştırılacaktır.

1.2. Tasarım Hedefleri

Aşağıdaki yedi tasarım hedefi, önceki fazlarda tanımlanan fonksiyonel ve fonksiyonel olmayan gereksinimlerden ve Proje Kısıt ve Etkiler (PKE) dokümanında belgelenen kısıtlardan türetilmiştir. Her hedef, mimari kararları yönlendiren ve ölçülebilir bir göstergeye bağlanan bir tasarım ilkesi olarak ele alınmıştır.

1.2.1. Hedeflerin Tanımı

Kod	Tasarım Hedefi	Tanım ve Mimari Karşılığı	Ölçülebilir Gösterge
TH1	Erişilebilirlik ve sıfır maliyet	Sistemin tamamı ücretsiz ve açık kaynaklı bileşenlerden kurulur; lisanslı yazılım, ücretli bulut hizmeti veya ticari veritabanı kullanılmaz. Bu hedef, hedef kitlenin (belediyeler, bağımsız araştırmacılar) sisteme erişebilmesinin ön koşuludur.	Ticari lisanslı veya kapalı kaynak bağımlılık sayısı = 0
TH2	Kullanılabilirlik ve görsel öncelikli etkileşim	Şehir plancısı profilindeki bir kullanıcı, XML dosyası düzenlemeden ve komut satırı kullanmadan, yalnızca harita üzerindeki nesnelerle etkileşerek senaryo tanımlayabilmelidir. Mimari, tüm teknik karmaşıklığı sunucu tarafında soyutlar.	Senaryo tanımlama akışı (yol topolojisi değiştirme eylemleri) ≤ 3 kullanıcı etkileşimi
TH3	Veri bütünlüğü ve tekrarlanabilirlik	Temel ağ hiçbir koşulda değiştirilemez; her senaryo bağımsız bir çalışma kopyası üzerinde üretilir ve motora yalnızca geçerli topolojiye sahip çıktılar iletilir. Aynı girdilerle yapılan koşumlar tekrar üretilebilir olmalıdır.	Koşum sonrası temel ağ dosyasının sağlaması (checksum) değişmez
TH4	Yanıt verebilirlik ve performans	Uzun süren hesaplama, kullanıcı arayüzünü bloklamamalıdır. Simülasyon asenkron bir iş olarak yürütülür; kullanıcıya anında geri bildirim verilir; sonuç görselleştirmesi tarayıcıyı kilitlemez.	(≥4 Core, 8GB RAM) ve 40-50 segmentlik ağda koşum ≤ 180 sn, sonuç render ≤ 3 sn, hata mesajı ≤ 2 sn
TH5	Taşınabilirlik ve kolay kurulum	Sistem, işletim sistemi bağımsız biçimde ve kaynak kodu değişikliği gerektirmeden ayağa kaldırılabilmelidir. Bileşenler konteynerlenir ve tek komutla çalıştırılır.	Linux ve Windows üzerinde başarılı kurulum
TH6	Bilimsel doğrulanabilirlik	Çıktılar gerçek dünya ölçümleriyle karşılaştırılabilir olmalı; her koşumun hangi girdi ve varsayımlarla üretildiği geriye dönük denetlenebilmelidir.	GEH < 10.0 oranı ≥ %65; OD benzerliği ≥ %80
TH7	Basitlik ve sürdürülebilirlik	Mimari, öğrenci ekibinin bakım kapasitesini aşmayacak biçimde sade	Durum tutan zorunlu altyapı servisi yalnızca

Kod	Tasarım Hedefi	Tanım ve Mimari Karşılığı	Ölçülebilir Gösterge
		tutulur; yüksek işletim maliyeti yaratan altyapı bileşenlerinden kaçınılır.	mesaj kuyruğudur; ek veritabanı veya arama motoru gibi servisler kullanılmaz.

Kaynak: Kendi Çalışmamız

1.2.2. Çatışan Tasarım Hedefleri ve Tercih Gerekçeleri

Yukarıdaki hedeflerin bir bölümü birbiriyle doğrudan çatışmaktadır. Bir mimarinin niteliği, bu çatışmaların gizlenmesiyle değil, hangi hedefin hangi gerekçeyle önceliklendirildiğinin açıkça beyan edilmesiyle ölçülür. Aşağıda, tasarım sürecinde karşılaşılan altı temel çatışma ve verilen kararlar sunulmaktadır.

Ç1. Doğruluk (TH6) ↔ Performans (TH4). Mikroskobik simülasyon ve Dinamik Kullanıcı Dengesi (DUA) iterasyonları, hesaplama maliyeti ağ büyüklüğü ve iterasyon sayısıyla hızla artan yöntemlerdir. Teorik denge noktasına ulaşana kadar iterasyon yapmak en doğru sonucu verir; ancak bu, koşum süresini öngörülemez kılar.

Tercih: performans lehine sınırlandırma yapılmıştır. Pilot bölge 40-50 segmentle sınırlanmış, DUA iterasyonlarına yapılandırılabilir bir üst sınır konmuş ve ardışık iterasyonlar arasındaki rota değişim oranı bir eşiğin altına düştüğünde iterasyon erken durdurulmuştur.

Gerekçe: sistem bir trafik tahmin motoru değil, karşılaştırmalı bir karar destek aracıdır; iki senaryo arasındaki **göreli** farkın doğru yakalanması, mutlak değerlerin son kırıntısına kadar hesaplanmasından daha önemlidir. Ayrıca öngörülebilir koşum süresi, kullanılabilirliğin (TH2) ön koşuludur.

Ç2. Görselleştirme zenginliği (TH2) ↔ İstemci performansı (TH4). Her aracın hareketini haritada canlandırmak sezgisel olarak çekicidir; ancak bunun için gereken mikroskobik çıktı dosyası (fcd-output.xml) her aracın her saniyedeki konumunu içerdiğinden gigabayt ölçeğine ulaşabilir ve sıradan bir tarayıcıda bellek taşmasına yol açar.

Tercih: araç bazlı animasyon yerine, sunucu tarafında toplulaştırılmış **segment bazlı ısı haritası** temel görselleştirme biçimi olarak benimsenmiştir. Araç hareketleri yalnızca örneklenmiş bir tekrar oynatma (replay) olarak, ikincil bir özellik biçiminde sunulur.

Gerekçe: karar destek için gereken bilgi (hangi segment ne kadar yoğun) makroskobik özet düzeyinde zaten mevcuttur; dosya boyutu veya ayrıştırma süresi kabul edilemez düzeye çıkarsa fcd çıktısı tamamen devre dışı bırakılıp yalnızca edgedata çıktısı kullanılacaktır.

Ç3. Basitlik (TH7) ↔ Ölçeklenebilirlik ve eşzamanlılık. İlişkisel bir veritabanı ve çok kullanıcılı bir oturum yönetimi, sistemi kurumsal ölçekte kullanılabilir kılardı; ancak kurulum, bakım ve şema yönetimi maliyeti getirirdi.

Tercih: dosya tabanlı kalıcılık ve tek kullanıcı / tek aktif senaryo modeli benimsenmiştir (bkz. Bölüm 3.4).

Gerekçe: sistemin veri profili ilişkisel değildir; ağ ve rota verileri zaten SUMO'nun doğal biçimi olan XML dosyalarıdır ve bu araçlar yalnızca dosya sisteminden okuyup yazar. Veritabanı eklemek, veriyi kullanan araçların hiçbirine fayda sağlamadan bir katman daha ekleyecekti. Bunun bedeli, eşzamanlı çok kullanıcılı kullanımın desteklenmemesidir ve bu bedel bilinçli olarak kabul edilmiştir.

Ç4. Kullanıcı özgürlüğü (TH2) ↔ Ağ bütünlüğü (TH3). Kullanıcıya harita üzerinde serbest yol çizme yetkisi vermek esneklik sağlar; ancak coğrafi bilgi sistemleri düzeyinde geometrik tutarsızlıklara, kopuk bağlantılara ve simülasyon motorunun çökmesine yol açar.

Tercih: serbest çizim yerine, mevcut düğüm ve kenarlar üzerinde çalışan, altı tipten oluşan kapalı bir **müdahale kataloğu** tanımlanmıştır.

Gerekçe: kısıtlı ama her zaman geçerli bir işlem kümesi, sınırsız ama sık sık geçersiz sonuç üreten bir arayüze tercih edilmiştir; bu tercih aynı zamanda projenin literatürden ayrışan özgün tasarım kararlarından biridir.

Ç5. Gerçek zamanlı izleme (TH2) ↔ Kararlılık ve kapsam (TH7). SUMO'nun TraCI arayüzü, simülasyon sırasında adım adım veri okumaya ve canlı görselleştirmeye izin verir. Bu, arayüzü daha etkileyici kılardı.

Tercih: TraCI kapsam dışında bırakılmış; SUMO yalnızca dosya tabanlı bir alt süreç olarak, toplu (batch) modda çalıştırılmıştır.

Gerekçe: TraCI, sunucu ile simülasyon arasında sürekli açık ve durum tutan bir bağlantı gerektirir; bu, asenkron iş kuyruğu mimarisiyle birlikte hata yönetimini belirgin biçimde karmaşıklaştırır. Canlı izleme, projenin karar destek amacı için gerekli değildir ve ileriye dönük bir genişleme seçeneği olarak bırakılmıştır.

Ç6. Kurulum kolaylığı (TH1, TH5) ↔ Erişim denetimi. Kimlik doğrulama ve kullanıcı yönetimi eklemek, sistemi paylaşımlı ortamlarda güvenli kılardı; ancak kurulum adımlarını artırır, kullanıcı hesabı verisi tutulmasını gerektirir ve kişisel veri işlememe ilkesiyle (KVKK) gerilim yaratır.

Tercih: varsayılan yerel dağıtımda kimlik doğrulama uygulanmamış; paylaşımlı sunucu dağıtımı için ters vekil sunucu (reverse proxy) düzeyinde opsiyonel bir koruma katmanı tanımlanmıştır (bkz. Bölüm 3.5).

Gerekçe: sistem kişisel veri tutmadığından ve tek rollü olduğundan, uygulama içi bir yetkilendirme modelinin sağlayacağı fayda, getireceği karmaşıklığı karşılamamaktadır.

1.3. Tanımlar, Kısaltmalar ve Kavramlar

Bu bölümde dokümanın tamamında kullanılan temel kavramlar tanımlanmıştır. Alan (trafik mühendisliği) terimlerinin genişletilmiş açıklamaları Bölüm 6'daki sözlükte yer almaktadır.

Kısaltma / Terim	Açıklama
HLD / LLD	Yüksek Seviye Tasarım (High-Level Design) / Düşük Seviye Tasarım (Low-Level Design).
PKE	Proje Kısıt ve Etkiler dokümanı; projenin standartlarını, kısıtlarını ve beklenen etkilerini tanımlayan önceki faz dokümanı.
TH	Tasarım Hedefi; mimari kararları yönlendiren yedi ilkeden her biri.
AS	Tek bir sorumluluk eksenine sahip mimari bileşen.
TraCI	Traffic Control Interface-SUMO'nun, simülasyon çalışırken adım adım veri okuma ve yazmaya izin veren canlı (online) kontrol arayüzü; bu projede kullanılmamıştır.

Kısaltma / Terim	Açıklama
MK	Mimari Karar; özetlenen ve tasarım hedeflerinden türetilen altı temel mimari karardan her biri.
SUMO	Simulation of Urban MObility-açık kaynaklı mikroskobik trafik simülasyon motoru [1].
OSM	OpenStreetMap-açık harita veri kaynağı [2].
OD Matrisi	Başlangıç-Varış (Origin-Destination) matrisi; bölgeler arası seyahat talebini gösteren veri yapısı.
DUA	Dinamik Kullanıcı Dengesi (Dynamic User Assignment); sürücülerin ağa iteratif uyum sürecini modelleyen algoritma [9].
GEH	Gözlemlenen ve simüle edilen trafik hacimleri arasındaki farkı ölçen ampirik istatistik [4].
KL Iraksama	Kullback-Leibler ıraksama; iki olasılık dağılımı arasındaki farkı ölçen metrik [5].
Temel Ağ (Base Network)	OSM'den üretilen, hiçbir müdahaleyle değiştirilmeyen salt-okunur referans yol ağı (base.net.xml).
Çalışma Kopyası	Her senaryo için temel ağdan türetilen ve müdahalelerin uygulandığı bağımsız ağ dosyası (modified.net.xml).
Müdahale	Kullanıcının tanımladığı topolojik değişiklik; altı tipten oluşan kapalı bir katalogla sınırlıdır.
İş	Asenkron kuyrukta yürütülen tek bir simülasyon koşumu; kendi çalışma dizinine ve durum kaydına sahiptir.
Modüler Monolit	Tek bir dağıtım birimi olarak paketlenen, ancak iç sorumlulukları net sınırlarla ayrılmış mimari yaklaşım.
Yoklama (Polling)	İstemcinin, sunucudaki iş durumunu belirli aralıklarla sorgulaması.
Devre Kesici	Ardışık hatalarda alt sistemi geçici olarak devre dışı bırakan hata izolasyon örüntüsü (circuit breaker).
Artefakt	Sistem tarafından üretilen ve dosya sisteminde tutulan çıktı (ağ dosyası, rota dosyası, log, metrik JSON'u).
Kara Kutu	İç işleyişine müdahale edilmeden, yalnızca girdi/çıktı sözleşmesi üzerinden kullanılan bileşen.

Kaynak: Kendi Çalışmamız

2. MEVCUT YAZILIM MİMARİSİ

Proje, önceden geliştirilmiş bir yazılımın yeniden yapılandırılması değil, sıfırdan geliştirilen (greenfield) bir sistemdir; bu nedenle devralınan ve dönüştürülmesi gereken bir kurumsal mimari bulunmamaktadır. Bununla birlikte, tasarım kararlarının gerekçelendirilebilmesi için sistemin yerini alacağı mevcut çalışma biçimlerinin ve devralınan tek mimari unsurun tanımlanması gerekmektedir.

2.1. Mevcut Çalışma Biçimleri

Kentsel altyapı müdahalelerinin analizinde bugün iki yaklaşım hâkimdir. Birincisi, PTV Vissim ve Aimsun gibi kapalı kaynaklı ticari masaüstü yazılımlardır [6]. Bu araçlar "kalın istemci" mimarisiyle çalışır; yüksek işlem gücüne sahip iş istasyonları gerektirir, yüksek lisans maliyetleri taşır ve web tabanlı esnek erişim sunmaz. İkincisi, SUMO [1] ve MATSim [7] gibi açık kaynaklı simülasyon motorlarıdır. Bu araçlar trafik fiziğini modellemede güçlüdür, ancak birer son kullanıcı ürünü değil, komut satırı ve XML dosyaları üzerinden çalışan hesaplama motorlarıdır: yeni bir senaryo test etmek için ağ topolojisini barındıran büyük XML dosyalarının elle düzenlenmesi, terminal komutlarının çalıştırılması ve süreçleri yöneten betiklerin yazılması gerekir. Simülasyon çıktısı ise görsel bir rapor değil, ham log dosyalarıdır.

Bu iki kutup arasında, teknik olmayan bir karar vericinin doğrudan kullanabileceği erişilebilir bir ara çözüm bulunmamaktadır. Önerilen sistem tam olarak bu boşluğu hedeflemektedir: açık kaynaklı motorun hesaplama gücünü korurken, onun önüne yönetilen bir senaryo yaşam döngüsü ve görsel bir karar destek katmanı koymak.

2.2. Devralınan Mimari Unsur: SUMO'nun Dosya Tabanlı Süreç Modeli

Sistemin mimarisini en çok kısıtlayan unsur, çekirdek olarak benimsenen SUMO'nun kendi mimarisidir ve bu unsur olduğu gibi devralınmıştır. SUMO araç seti; netconvert (ağ dönüştürme ve doğrulama), od2trips (OD matrisinden seyahat üretimi), duarouter (rota hesaplama ve DUA iterasyonu) ve sumo (mikroskobik koşum) olmak üzere birbirinden bağımsız çalıştırılabilir komut satırı programlarından oluşur. Bu programlar arasında paylaşılan tek arayüz dosya sistemidir: her araç dosyadan okur, dosyaya yazar ve süreç sonlanır.

Bu durumun mimari sonuçları doğrudandır ve önerilen tasarımın birçok kararını belirlemiştir:

- Entegrasyon, kütüphane çağrısı olarak değil, alt süreç (subprocess) çağrısı olarak gerçekleşir; dolayısıyla sistemde bir süreç yönetimi ve zaman aşımı denetimi katmanı zorunludur.
- Veri alışverişi dosya üzerinden yapıldığından, bileşenlerin paylaşılan bir dosya alanına erişmesi gerekir; bu, dağıtım topolojisinde ortak bir veri hacmi (volume) olarak karşılık bulur.
- Koşum süreleri dakikalar mertebesine ulaşabildiğinden, çağrıların bir web isteği içinde senkron biçimde beklenmesi mümkün değildir; asenkron iş kuyruğu kaçınılmazdır.
- SUMO'nun kendi dosya biçimleri (.net.xml, .rou.xml) sistemin fiilî veri modelini oluşturur; bu da kalıcılık katmanının neden dosya tabanlı tasarlandığının temel gerekçelerinden biridir (bkz. Bölüm 3.4).

Sistem, SUMO'yu bir kara kutu olarak ele alır: motorun iç algoritmalarına (araç takibi, şerit değiştirme modelleri) müdahale edilmez, yalnızca girdi dosyaları hazırlanır ve çıktı dosyaları yorumlanır. Bu

sınır, hem kapsamı korumakta hem de motorun ileride sürüm yükseltmesi durumunda sistemin etkilenmesini asgariye indirmektedir.

3. ÖNERİLEN YAZILIM MİMARİSİ

3.1. Genel Bakış

Önerilen sistem, **katmanlı bir modüler monolit** ile **kuyruk tabanlı asenkron işçi (worker)** yaklaşımını birleştiren bir mimariye sahiptir. Sistem üç çalıştırılabilir birim olarak dağıtılır: sunum (statik web uygulaması), orkestrasyon (API sunucusu) ve simülasyon işçisi. Ancak bu birimler bağımsız olarak sürümlenen ve ayrı ayrı ölçeklenen servisler değildir; ortak bir kod tabanını ve ortak bir veri alanını paylaşırlar.

Bu ayrımın açıkça belirtilmesi önemlidir: **sistem bir mikroservis mimarisi değildir ve öyle olduğu iddia edilmemektedir.** Mikroservis yaklaşımı; bağımsız dağıtım, ayrı veri sahipliği ve servisler arası ağ protokolleri gerektirir. Buna karşılık simülasyon işçisinin ayrı bir süreç olarak çalıştırılması **teknik bir zorunluluktur**: Dakikalarca süren, CPU'ya bağımlı bir alt süreç, API sunucusunun istek döngüsünü bloklamamalıdır. Dolayısıyla mimari, "iş yükü ayrıştırması yapan modüler monolit" olarak tanımlanmaktadır.

Sistem mantıksal olarak beş katmandan oluşur:

- **Sunum katmanı**: Harita tabanlı etkileşim, senaryo tanımlama ve sonuç görselleştirme.
- **Orkestrasyon katmanı**: REST uçları, senaryo yaşam döngüsü, iş gönderimi ve durum yönetimi.
- **Alan hizmetleri katmanı**: Ağ yönetimi, talep modelleme ve analiz/raporlama işlevleri.
- **Entegrasyon katmanı**: SUMO araç setinin alt süreç olarak çalıştırılması ve çıktıların toplanması.
- **Kalıcılık katmanı**: Dosya tabanlı ortak veri deposu.

Mimarinin temelini oluşturan altı karar aşağıda özetlenmiştir. Bu kararlar, Bölüm 1.2.2'de tartışılan çatışmaların çözümünden doğmuştur ve dokümanın geri kalanında ayrıntılandırılmaktadır.

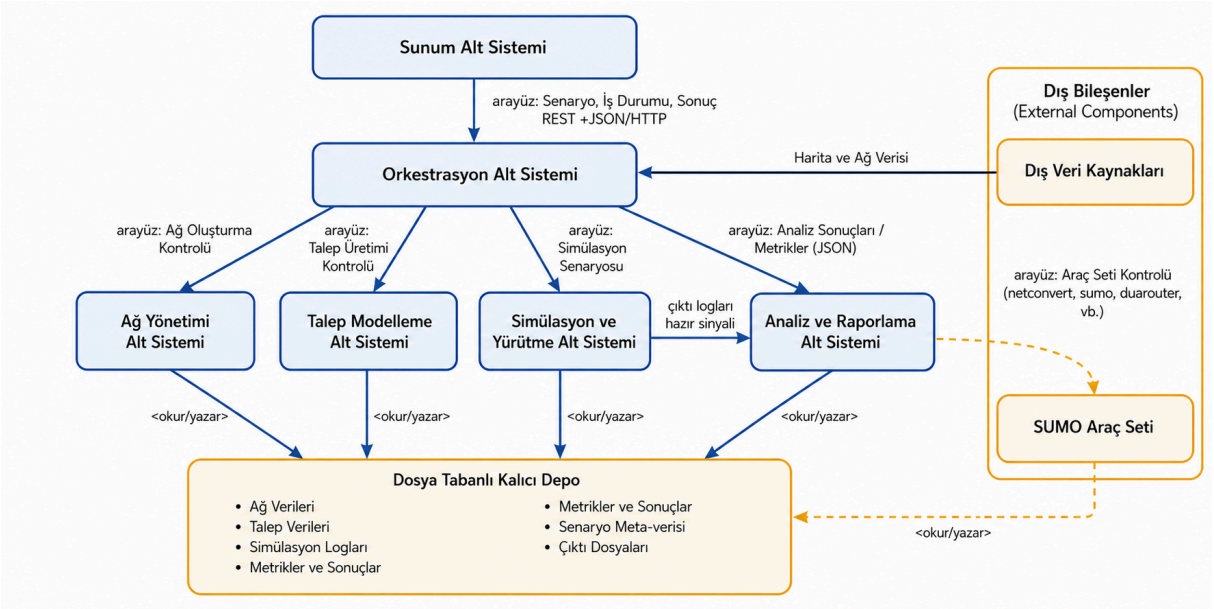
Kod	Mimari Karar	Gerekçe	İlgili Hedef
MK1	SUMO kara kutu olarak, dosya tabanlı alt süreç çağrılarıyla entegre edilir. [1]	Motorun süreç modeliyle uyum; asenkron mimaride hata yönetiminin sadeleşmesi; kapsamın korunması.	TH7, Ç5
MK2	Temel ağ değiştirilemez; her senaryo bağımsız bir çalışma kopyası üzerinde üretilir.	Tekrarlanabilir üretim ve veri bütünlüğü; senaryoların birbirini kirletmemesi.	TH3, Ç4

MK3	Ağ doğrulaması senkron bir kapı olarak, kuyruğa almadan önce yapılır.	Geçersiz girdinin kaynak tüketmeden hızlı reddi; kullanıcıya anında geri bildirim.	TH3, TH4
MK4	Simülasyon koşumu asenkron iş kuyruğu üzerinden yürütülür; istemci yoklama ile durum sorgular.	Uzun süren işin istek döngüsünü bloklamaması; arayüzün kilitlenmemesi.	TH4
MK5	Kalıcılık dosya tabanlıdır; ilişkisel veya NoSQL veritabanı kullanılmaz.	Verinin doğası ilişkisel değildir; SUMO araçları yalnızca dosyayla çalışır; kurulum ve bakım maliyeti düşer.	TH1, TH5, TH7, Ç3
MK6	Ham mikroskobik çıktı istemciye gönderilmez; sunucuda toplulaştırılarak ısı haritasına dönüştürülür.	Tarayıcı bellek sınırının aşılmaması; veri transfer boyutunun düşürülmesi.	TH4, Ç2

Kaynak: Kendi Çalışmamız

3.2. Alt Sistem Ayrışımı

Sistem, sorumlulukları birbirinden ayrılmış altı alt sisteme ve bunların ortaklaşa kullandığı bir kalıcı veri deposuna ayrıştırılmıştır. Ayrıştırmanın temel ilkesi, **her alt sistemin tek bir sorumluluk eksenine sahip olması** ve alt sistemler arası bağımlılıkların tek yönlü tutulmasıdır: sunum yalnızca orkestrasyonu tanır, orkestrasyon alan hizmetlerini çağırır, alan hizmetleri birbirini doğrudan çağırmaz. Bu kural, bir alt sistemdeki değişikliğin diğerlerine yayılmasını sınırlar ve LLD aşamasında modüllerin paralel geliştirilebilmesini sağlar.



Şekil 1. Alt sistem ayrışımı - UML bileşen diyagramı

3.2.1. Alt Sistemlerin Sorumlulukları

AS1 - Sunum Alt Sistemi: Kullanıcının sistemle kurduğu tüm etkileşimi yönetir. Harita üzerinde yol segmenti ve kavşak seçimi, müdahale kataloğunun sunulması, seçilen müdahalelerin bir "sepet" mantığıyla biriktirilmesi, koşumun tetiklenmesi, iş durumunun düzenli aralıklarla yoklanması ve sonuçların ısı haritası, karşılaştırma tablosu ile zaman serisi grafikleri biçiminde görselleştirilmesi bu alt sistemin sorumluluğundadır. Ayrıca sonuç ekranlarında, çıktıların model temelli tahminler olduğunu belirten kalıcı bilgilendirme metni bu katmanda sunulur.

Kapsam dışı: Hiçbir simülasyon, doğrulama veya metrik hesaplaması istemci tarafında yapılmaz; sunum katmanı yalnızca kendisine hazır biçimde iletilen veriyi çizer.

AS2 - Orkestrasyon Alt Sistemi: Sistemin denetim merkezidir. İstemciden gelen istekleri karşılar, senaryo meta-data'sını yönetir, simülasyon öncesi hazırlık adımlarını (çalışma kopyasının oluşturulması, doğrulamanın tetiklenmesi, talep artefaktlarının hazır olduğunun kontrolü) sıraya koyar, doğrulama başarılıysa işi kuyruğa gönderir ve iş durumu sorgularını yanıtlar.

Kapsam dışı: Simülasyonu kendisi çalıştırmaz; uzun süren hiçbir hesaplamayı istek döngüsü içinde yapmaz.

AS3 - Ağ Yönetimi Alt Sistemi: Yol ağının yaşam döngüsünden sorumludur ve üç işlevi kapsar: Kurulum aşamasında OSM [2] verisinin OSMnx [8] çekilerek netconvert ile temel ağa dönüştürülmesi, Her senaryoda temel ağdan bağımsız bir çalışma kopyası üretilmesi ve müdahalelerin bu kopyaya işlenmesi, Üretilen kopyanın bağlantısallık, izole düğüm ve rotalanabilirlik açısından doğrulanması.

Kapsam dışı: Temel ağ dosyasını hiçbir koşulda değiştirmez; bu, mimarinin bir kuralıdır.

AS4 - Talep Modelleme Alt Sistemi: Trafik talebinin sentetik olarak üretilmesinden sorumludur. Ham trafik hacim verisi ile arazi kullanım verisini temizler, saatlik hacim profillerini çıkarır, Çekim Modeli (Gravity Model) [9] ile OD matrisini üretir ve od2trips aracılığıyla seyahat tanımlarına dönüştürür.

Kapsam dışı: Rota hesaplaması yapmaz; rotalama, simülasyon yürütme alt sisteminin duarouter çağrısıyla gerçekleşir. Talep artefaktları senaryodan bağımsızdır ve birden çok koşumda yeniden kullanılabilir.

AS5 - Simülasyon Yürütme Alt Sistemi: Asenkron koşumun tek sorumlusudur. İş kuyruğundan aldığı görevi kendi izole çalışma dizininde yürütür: Önce duarouter ile rota dosyasını (DUA iterasyonları dâhil) üretir, ardından sumo ikilisini çalıştırarak çıktı loglarını oluşturur. Alt süreçlerin zaman aşımı denetimi, iptali ve durum raporlaması bu alt sisteme aittir.

Kapsam dışı: Çıktıları yorumlamaz; ham log dosyalarını üretip işi tamamlanmış olarak işaretler.

AS6 - Analiz ve Raporlama Alt Sistemi: Ham çıktıyı karar destek bilgisine dönüştürür. Çıktı XML dosyalarını ayrıştırır; segment düzeyinde yoğunluk, ortalama hız ve araç hacmini, kavşak düzeyinde ortalama gecikme ve kuyruk uzunluğunu hesaplar; temel senaryo ile müdahale senaryosunu karşılaştırır; tanımlı eşikler aşıldığında kural tabanlı darboğaz uyarılarını üretir ve tüm bunları arayüzün tüketebileceği tek bir JSON yapısına dönüştürür.

Kapsam dışı: Görselleştirmeyi kendisi yapmaz; yalnızca görselleştirmeye hazır veriyi üretir.

Dosya Tabanlı Kalıcı Depo: Bir alt sistem değil, tüm alt sistemlerin paylaştığı bir kaynaktır. Temel ağ, talep artefaktları, senaryo meta-verileri, çalışma kopyaları, iş kayıtları, ham çıktılar ve hesaplanmış metrikler burada tutulur. Erişim kuralları ve dizin düzeni Bölüm 3.4'te tanımlanmıştır.

3.2.2. Alt Sistem - Katman eşlemesi

Aşağıdaki tablo, Bölüm 3.1'de tanımlanan beş mimari katman ile Bölüm 3.2.1'de tanımlanan alt sistemler arasındaki ilişkiyi göstermektedir. Katmanlar mimarinin soyutlama seviyesine göre yatay bölünmesini, alt sistemler ise işlevsel sorumluluğa göre dikey bölünmesini temsil eder.

Katman	İlgili Alt Sistemler
Sunum Katmanı	AS1 - Sunum Alt Sistemi
Orkestrasyon Katmanı	AS2 - Orkestrasyon Alt Sistemi
Alan Hizmetleri Katmanı	AS3 - Ağ Yönetimi AS4 - Talep Modelleme AS6 - Analiz ve Raporlama
Entegrasyon Katmanı	AS5 - Simülasyon Yürütme
Kalıcılık katmanı	Dosya Tabanlı Kalıcı Depo

Not: Dosya tabanlı kalıcı depo tam olarak bir alt sistem değildir, tüm alt sistemlerin paylaştığı bir kaynak olarak iş görür.
Kaynak: Kendi çalışmamız

3.2.3. Alt Sistemler Arası Arayüzler

Alt sistemler arasındaki bağımlılıklar, aşağıdaki mantıksal arayüzler üzerinden kurulmuştur. Arayüzler bu aşamada yalnızca sorumluluk sözleşmesi düzeyinde tanımlanmıştır; imza, parametre ve veri yapısı ayrıntıları LLD dokümanında kesinleştirilecektir.

Arayüz	Sağlayan	Tüketen	Etkileşim Tipi	Sözleşme Özeti
Senaryo	AS2	AS1	REST / JSON	Senaryo oluşturma, müdahale ekleme/çıkarma, aktif senaryonun okunması.
İşDurumu	AS2	AS1	REST / JSON	Koşum tetikleme; iş kimliği ve durum bilgisinin döndürülmesi.
Sonuç	AS2	AS1	REST / JSON	Tamamlanmış işin metrik, ısı haritası verisi ve uyarılarının döndürülmesi.
AğKopyalama	AS3	AS2	Süreç içi çağrı	Temel ağdan çalışma kopyası üretimi ve müdahalelerin uygulanması.
AğDoğrulama	AS3	AS2	Süreç içi çağrı	Çalışma kopyasının geçerliliği; başarısızlıkta gerekçe kodu döndürülür.
TalepÜretimi	AS4	AS2	Süreç içi çağrı	Seçilen zaman aralığı için talep artefaktlarının hazırlanması/doğrulanması.

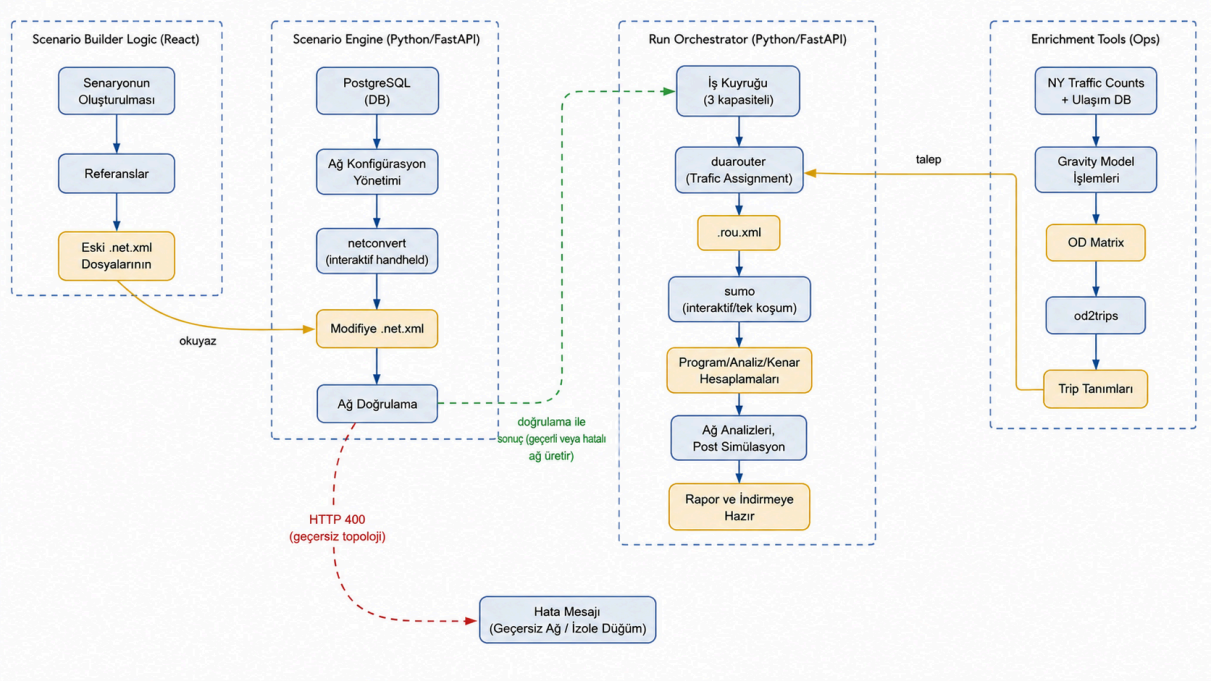
Arayüz	Sağlayan	Tüketen	Etkileşim Tipi	Sözleşme Özeti
İşKuyruğu	AS5	AS2	Mesaj kuyruğu (asenkron)	İşin kuyruğa alınması ve durum geçişlerinin bildirilmesi.
MetrikSonuç	AS6	AS2	Süreç içi çağrı	Ham çıktılardan metrik, karşılaştırma ve uyarı kümesinin üretilmesi.
SUMO CLI	Dış yazılım	AS3, AS4, AS5	Alt süreç (dosya tabanlı)	netconvert, od2trips, duarouter ve sumo araçlarının çağrılması.
Kalıcı Depo	Dosya Tabanlı Kalıcı Depo	AS3, AS4, AS5, AS6	Dosya sistemi (senkron okuma/yazma)	Temel ağ, talep artefaktları, çalışma kopyaları ve ham çıktıların; paylaşılan dizin düzeni üzerinden okunması/yazılması.

Kaynak: Kendi Çalışmamız

3.2.4. Alt Sistemler Arası Veri Akışı

Sistemdeki veri akışı, birbirinden farklı tetikleyicilere sahip dört akışa ayrılmaktadır. Bu ayrım, hangi işlemin ne sıklıkta ve hangi maliyetle çalıştığını netleştirdiği için mimari açıdan belirleyicidir.

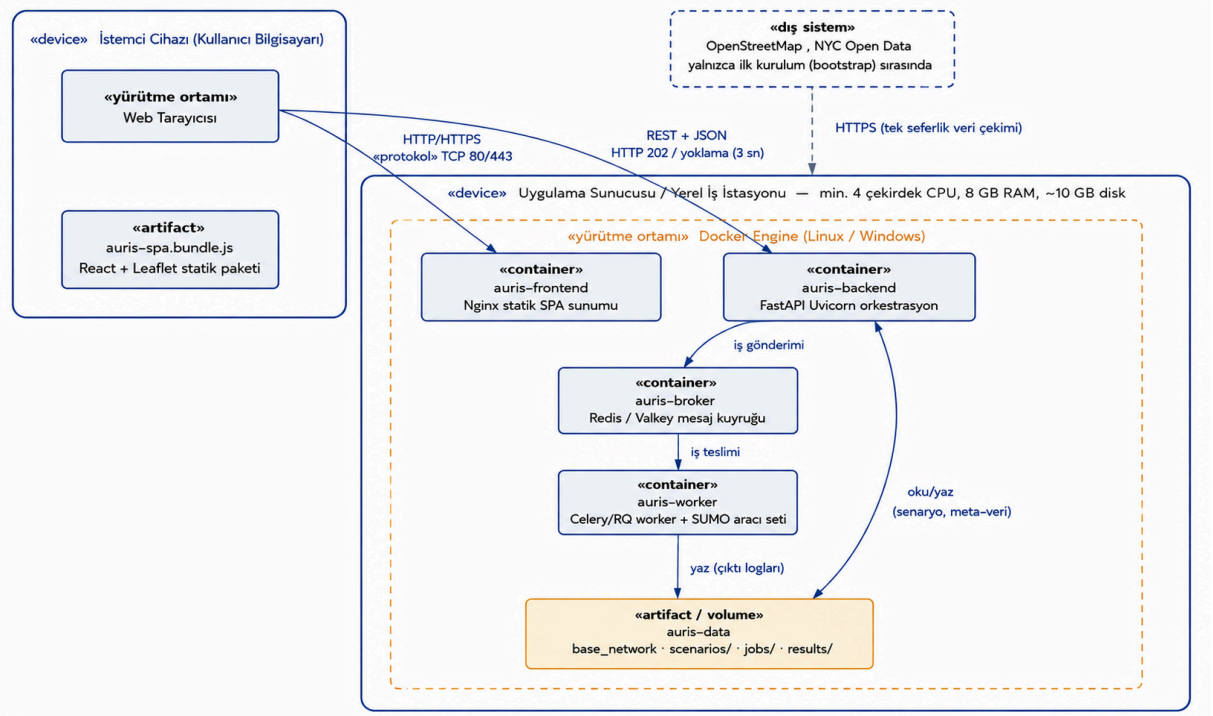
- **Kurulum "bootstrap" akışı (tek seferlik):** OSM verisi çekilir, netconvert ile temel ağ üretilir. Bu akış yalnızca pilot bölge ilk kez tanımlanırken çalışır ve ürettiği temel ağ bundan sonra salt okunur kabul edilir.
- **Talep hazırlama akışı (seyrek):** Trafik hacim ve arazi kullanım verilerinden Çekim Modeli ile OD matrisi ve seyahat tanımları üretilir. Talep artefaktları senaryolardan bağımsız olduğundan, aynı zaman aralığı için bir kez üretilip birçok koşumda yeniden kullanılır.
- **Senaryo üretim akışı (her koşumda):** Kullanıcının müdahale listesi temel ağın bir kopyasına uygulanır, netconvert ile bağlantılar yeniden hesaplanır ve ortaya çıkan çalışma kopyası doğrulanır. Bu akış senkrondur ve hızlı sonuçlanmalıdır.
- **Simülasyon ve analiz akışı (asenkron):** Doğrulanmış ağ ve talep artefaktları ile duarouter ve sumo çalıştırılır; üretilen loglar ayrıştırılarak metriklere ve ısı haritası verisine dönüştürülür.



Şekil 2. Uçtan uca veri akışı ve artefakt üretimi

3.3. Donanım / Yazılım Eşlemesi

Sistem, tek bir fiziksel makine üzerinde çalışacak biçimde tasarlanmıştır. Hedef kitlenin kurumsal sunucu veya ücretli bulut altyapısına erişimi bulunmadığı varsayıldığından (TH1, TH5), dağıtım birimi olarak **konteyner** seçilmiş ve tüm bileşenler tek bir düzenleme dosyasıyla ayağa kaldırılabilir hâle getirilmiştir. Referans donanım, Spesifikasyon Raporu'nda tanımlanan asgari yapılandırmadır: en az 4 çekirdek CPU, 8 GB RAM ve yaklaşık 10 GB boş disk alanı.



Şekil 3. Donanım/yazılım eşlemesi-UML dağıtım diyagramı

3.3.1. Dağıtım Birimleri

Dağıtım Birimi	Barındırdığı Alt Sistem	Temel Yazılım Yığını	Kaynak Profili
auris-frontend «container»	AS1-Sunum	Nginx üzerinde statik olarak sunulan React [11] + Leaflet [12] paketi	Düşük: ~128 MB bellek; CPU kullanımı ihmal edilebilir.
auris-backend «container»	AS2-Orkestrasyon AS3, AS4, AS6 (süreç içi)	Python 3 · FastAPI [10] · Uvicorn · OSMnx, Pandas, NumPy · SUMO (netconvert, od2trips)	Orta: ~1 GB bellek; kısa süreli CPU yoğunluğu (doğrulama ve ayrıştırma sırasında).
auris-worker «container»	AS5-Simülasyon Yürütme	Python 3, görev kuyruğu işçisi (Celery veya RQ), SUMO araç seti (duarouter, sumo)	Yüksek: CPU'ya bağımlı; koşum süresince çekirdeklerin çoğunu kullanır, ~2-4 GB bellek.
auris-broker «container»	Altyapı (iş kuyruğu)	Redis veya BSD lisanslı çatalı Valkey	Düşük: ~64 MB bellek; yalnızca mesaj aktarımı.
auris-data «volume»	Kalıcı depo (paylaşımlı)	Ana makine dosya sisteminde bağlanan (mount) veri hacmi	Disk: pilot bölge ölçeğinde koşum başına yüzlerce MB; toplamda birkaç GB.
Web tarayıcısı «yürütme ortamı»	AS1'in çalıştığı istemci ortamı	Modern bir web tarayıcısı	Düşük-orta: ısı haritası çizimi sırasında geçici bellek kullanımı.

Kaynak: Kendi Çalışmamız

Lisans notu [13]: iş kuyruğu bileşeni için Redis'in güncel sürümlerinde lisans modeli değiştiğinden, projenin "yalnızca OSI onaylı açık kaynak bileşen kullanılır" kısıtına (TH1) tam uyum sağlamak amacıyla BSD lisanslı Valkey çatalı eşdeğer bir seçenek olarak değerlendirilmektedir. Her iki seçenek de aynı protokolü kullandığından mimari üzerinde etkisi yoktur.

3.3.2. Dağıtım Senaryoları ve Taşınabilirlik

İki dağıtım senaryosu öngörülmüştür. **Varsayılan senaryoda** tüm konteynerler kullanıcının kendi bilgisayarında çalışır; sistem çevrimdışı bir analiz aracı gibi kullanılır ve dış ağ erişimi yalnızca ilk kurulumda (harita ve veri setlerinin indirilmesi için) gereklidir. **Kurum içi paylaşımlı senaryoda** aynı konteyner kümesi bir sunucuda çalıştırılır ve kullanıcılar tarayıcı üzerinden erişir; bu senaryoda önüne bir reverse proxy yerleştirilerek TLS sonlandırma ve isteğe bağlı erişim koruması sağlanır (bkz. Bölüm 3.5.2).

Taşınabilirlik, konteyner imajlarının işletim sistemi bağımsız olmasıyla sağlanır: aynı düzenleme dosyası Linux ve Windows üzerinde kaynak kodu değişikliği gerektirmeden çalışır. SUMO araç setinin işletim sistemine özgü kurulum adımları imajın içine kapsüllendiğinden, kullanıcıdan SUMO

kurulumu beklenmez. Bu, hedef kitle açısından kurulum eşiğini belirgin biçimde düşüren bir tasarım kararıdır.

3.4. Kalıcı Veri Yönetimi

3.4.1. Veri Profilinin Analizi

Depolama teknolojisi seçimi, verinin niteliğinden türetilmiştir. Sistemin ürettiği ve tükettiği veri şu özellikleri taşır:

- **Büyük ve hiyerarşik dosyalar:** ağ (.net.xml) ve rota (.rou.xml) dosyaları binlerce iç içe geçmiş öğe içeren XML belgeleridir; satır bazlı kayıtlar değildir.
- **Bir kez yaz, çok kez oku:** Üretilen artefaktlar sonradan güncellenmez; bir koşum tamamlandığında çıktıları değişmez kabul edilir.
- **İlişkisel sorgu ihtiyacının yokluğu:** Sistemde birleştirme (join), toplulaştırma sorgusu veya raporlama sorgusu gerektiren bir kullanım senaryosu bulunmamaktadır; erişim her zaman bilinen bir kimlik üzerinden doğrudan okumadır.
- **Tek yazar, düşük eşzamanlılık:** Tek kullanıcı ve tek aktif senaryo modeli gereği, aynı kayda eşzamanlı yazma durumu oluşmaz; dolayısıyla işlem (transaction) ve kilitleme ihtiyacı yoktur.
- **Araç uyumu zorunluluğu:** SUMO araçları veriyi yalnızca dosya sisteminden okuyabilir. Veri bir veritabanında tutulsaydı, her koşumdan önce dosyaya geri yazılması gerekirdi.

3.4.2. Depolama Teknolojisi Seçimi

Aşağıdaki tabloda değerlendirilen alternatifler ve karar gerekçeleri sunulmaktadır.

Seçenek	Güçlü Yönü	Bu Proje İçin Sakıncası	Karar
PostgreSQL (ilişkisel)	ACID işlemler, güçlü sorgu ve birleştirme yetenekleri, eşzamanlı çok kullanıcı desteği.	Sistemde ne işlem bütünlüğü ne de karmaşık sorgu ihtiyacı vardır. XML artefaktları yine dosyada tutulmak zorunda olduğundan veritabanı yalnızca meta-veri taşır; buna karşılık kurulum, şema göçü ve bakım yükü getirir. Sıfır bütçe ve kolay kurulum hedefleriyle çelişir.	Reddedildi
MongoDB (belge tabanlı)	Şemasız JSON belgeleri; senaryo meta-verisiyle biçimsel uyum.	Sağladığı esneklik, düz JSON dosyalarının sunduğunun ötesine geçmemektedir. Ek bir sunucu süreci ve konteyner gerektirir; sistemin toplam bileşen sayısını gerekçesiz artırır.	Reddedildi
SQLite (gömülü)	Sunucu süreci gerektirmez, tek dosyadır, taşınabilirdir; iş kayıtları için uygun olabilir.	Gerçek bir fayda yalnızca çok sayıda iş kaydı üzerinde sorgu yapılması hâlinde ortaya çıkar. Mevcut kapsamda iş sayısı düşüktür ve erişim kimlik üzerinden doğrudandır.	Kapsam dışı; ileriye dönük yükseltme yolu olarak saklandı

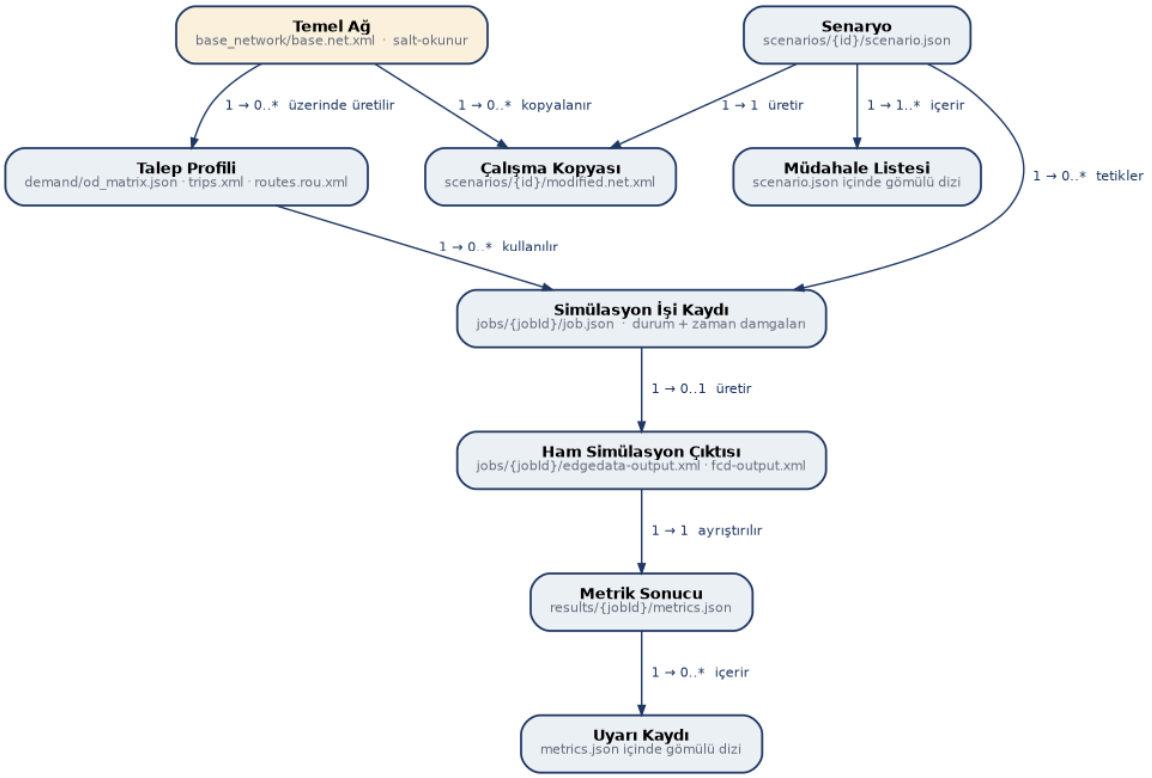
Seçenek	Güçlü Yönü	Bu Proje İçin Sakıncası	Karar
Dosya tabanlı (XML + JSON)	SUMO'nun doğal biçimiyle birebir uyum; sıfır kurulum; konteyner hacmiyle doğrudan taşınabilirlik ve yedeklenebilirlik; insan tarafından okunabilir ve sürüm kontrolüne alınabilir meta-veri.	Karmaşık sorgulama desteklenmez; eşzamanlı çok kullanıcılı yazma güvenli değildir. Her iki sınırlılık da mevcut kapsamda kabul edilmiştir.	**Seçildi**

Kaynak: Kendi Çalışmamız

Seçimin özeti şudur: veritabanı kullanmama kararı bir kolaycılık değil, **veri profiline dayanan bilinçli bir tercihtir**. Verinin kendisi zaten dosya biçimindedir, tüketicisi olan araçlar yalnızca dosyadan okumaktadır ve sistemin ilişkisel bir sorgu yüküne ihtiyacı yoktur. Bu koşullarda bir veritabanı katmanı eklemek, mimariye fayda sağlamadan işletim maliyeti ve hata yüzeyi eklerdi. Buna karşılık kabul edilen bedel açıktır: Sistem eşzamanlı çok kullanıcılı kullanıma uygun değildir ve geçmiş koşumlar üzerinde toplu analitik sorgulama yapılamaz.

3.4.3. Kavramsal Veri Şeması

Kalıcı veri, varlık düzeyinde aşağıdaki gibi modellenmiştir. Şema, dosya sistemindeki karşılıklarıyla birlikte verilmiştir; öznitelik ve davranış ayrıntıları LLD kapsamındadır.



Şekil 4. Kalıcı verinin kavramsal şeması (varlık düzeyi)

Şemanın iki kuralı mimari açıdan bağlayıcıdır. Birincisi, **Temel Ağ düğümü hiçbir ilişkide hedef değildir**; ondan yalnızca kopya türetilir, üzerine yazılmaz. İkincisi, **Simülasyon İşi ile Metrik Sonucu arasındaki ilişki 1 → 0..1'dir**; bir iş yalnızca başarıyla tamamlandığında sonuç üretir; kuyrukta, çalışırken veya başarısız durumdayken sonuç nesnesi bulunmaz. Arayüz bu ayrımı doğrudan yansıtır.

3.4.4. Dizin Düzeni ve Veri Yaşam Döngüsü

Paylaşımlı veri hacmi, sorumluluk sınırlarını dosya sistemi düzeyinde de görünür kılacak biçimde düzenlenmiştir:

Dizin	İçerik	Erişim Kuralı	Yaşam Süresi
base_network/	Temel ağ dosyası ve kaynak OSM özütü	AS3 yazar (yalnızca kurulumda); diğer tüm bileşenler salt-okunur bağlar	Kalıcı
demand/	OD matrisi, seyahat tanımları, hacim profilleri	AS4 yazar; AS5 okur	Kalıcı (zaman aralığı başına)
scenarios/{id}/	Senaryo meta-verisi (JSON) ve çalışma kopyası	AS2 ve AS3 yazar; AS5 okur	Kullanıcı silene kadar
jobs/{jobId}/	İş kaydı, alt süreç logları, ham çıktı XML'leri	AS5 yazar; AS6 okur	Geçici, otomatik temizlemeye tabi
results/{jobId}/	Hesaplanmış metrikler, ısı haritası verisi, uyarılar (JSON)	AS6 yazar; AS2 okur	Senaryo ömrü boyunca

Kaynak: Kendi Çalışmamız

Veri bütünlüğünü korumak amacıyla iki kural uygulanır. **Atomik yazma:** Üretilen her artefakt önce geçici bir ada yazılır ve yalnızca tamamlandığında nihai adına taşınır; böylece yarım kalmış bir dosya hiçbir zaman geçerli bir artefakt gibi okunamaz. **Otomatik temizleme:** İş çalışma dizinleri, tanımlı bir iş sayısı veya disk doluluk eşiği aşıldığında en eskiden başlayarak silinir; silme işlemi yalnızca `jobs/` dizinini etkiler, hesaplanmış sonuçları etkilemez.

3.4.5. Yedekleme ve Kurtarma Stratejisi

Yedekleme stratejisi, verinin **yeniden üretilebilirliğine** dayanmaktadır. Sistemdeki artefaktlar iki sınıfa ayrılır:

- **Birincil veri (yeniden üretilmesi maliyetli veya imkânsız):** temel ağ dosyası, işlenmiş talep artefaktları ve kullanıcının tanımladığı senaryo meta-verileri. Bu grup düzenli olarak yedeklenir.
- **Türetilmiş veri (her zaman yeniden üretilebilir):** çalışma kopyaları, ham çıktı logları ve hesaplanmış metrikler. Bu grup kaybedilirse ilgili senaryo yeniden koşularak elde edilir; yedeklenmesi zorunlu değildir.

Uygulama düzeyinde strateji üç unsurdan oluşur

1. **Hacim anlık görüntüsü**: Birincil veri kümesinin toplam boyutu küçüktür (pilot bölge ölçeğinde ~100 MB); bu nedenle harici veya ücretli bir yedekleme altyapısına ihtiyaç duyulmaz. Yedekleme, depo ile birlikte dağıtılan bir arşivleme betiği aracılığıyla tek komutla yapılır: `auris-data` hacminin birincil dizinleri (`base_network/`, `demand/`, `scenarios/`) düzenli aralıklarla arşivlenerek ana makinede (Docker'ın üzerinde çalıştığı bilgisayar) saklanır. Elde edilen arşiv, kullanıcının tercihine göre yerel diskte, taşınabilir bir bellekte veya kurumun kendi ağ paylaşımında saklanabilir; senaryo meta-verileri metin tabanlı olduğundan kod deposunda sürümlenerek de korunabilir.
2. **Senaryo dışa aktarımı**: Her senaryo, kendi meta-verisi ve müdahale listesiyle birlikte tek bir JSON dosyası olarak dışa aktarılabilir; bu dosya, temel ağ mevcut olduğu sürece senaryoyu bir başka kurulumda yeniden oluşturmaya yeter ve ekip içi paylaşımın da yolu budur.
3. **Konfigürasyonun sürüm kontrolü**: Konteyner düzenleme dosyaları, SUMO koşum parametreleri ve model eşikleri kod deposunda tutulur, dolayısıyla sistemin çalışır hâli her zaman yeniden kurulabilir.

Bu yaklaşımın sonucu şudur: Tam bir veri kaybı durumunda kurtarma, temel ağın ve talep artefaktlarının geri yüklenmesi ile senaryoların yeniden koşulmasından ibarettir. Sistem çevrimdışı bir analiz aracı olduğundan, kurtarma süresi kritik bir işletim gereksinimi değildir.

3.5. Erişim Denetimi ve Güvenlik

3.5.1. Tehdit Yüzeyinin Belirlenmesi

Sistemin güvenlik profili, iki özelliğiyle belirlenir. Birincisi, sistemde korunması gereken kişisel veri bulunmamaktadır: kullanıcı hesabı, kimlik bilgisi, konum geçmişi veya plaka gibi veriler ne toplanır ne işlenir. İkincisi, sistem kullanıcı girdisini işletim sistemi düzeyinde alt süreçlere aktarmaktadır. Bu ikinci özellik, sistemin **en belirgin** teknik tehdit yüzeyidir: arayüzden gelen bir müdahale tanımı, doğrudan veya dolaylı olarak netconvert, duarouter ve sumo çağrılarının parametrelerine ulaşır. Dolayısıyla güvenlik tasarımının ağırlık merkezi, kimlik doğrulama değil, girdi doğrulama ve süreç izolasyonudur.

3.5.2. Kimlik Doğrulama ve Yetkilendirme

Sistem tek rollü olarak tasarlanmıştır: tüm işlevleri kullanan tek aktör Trafik Planlayıcısı'dır; ikinci bir insan aktör (SUMO motoru sistem aktörüdür) bulunmadığından uygulama içi bir rol hiyerarşisine ihtiyaç duyulmaz. Bölüm 1.2.2'deki Ç6 çatışmasında açıklandığı üzere, varsayılan yerel dağıtımda uygulama düzeyinde kimlik doğrulama uygulanmaz; sistemin güvenlik sınırı, üzerinde çalıştığı bilgisayarın kendi oturum güvenliğidir.

Paylaşımlı sunucu dağıtımı seçildiğinde ise koruma, uygulama koduna dokunmadan altyapı düzeyinde eklenir: proxy server önce TLS sonlandırma yaparak istemci ile kendisi arasındaki tüm trafiği (HTTPS üzerinden) şifreler; bu şifreli kanal üzerinden HTTP Basic kimlik doğrulama veya API anahtarı denetimi uygulanır. Basic Auth kimlik bilgileri yalnızca Base64 ile kodlandığından kendi başına şifreleme sağlamaz; bu nedenle **TLS'siz bir bağlantıda kullanılması kabul edilmez**, mimaride Basic Auth her zaman TLS sonlandırmasından sonraki adımdır, asla onun yerine geçmez. Bu tasarım, güvenlik ihtiyacının dağıtım biçimine göre değiştiği gerçeğini kabul eder ve maliyeti yalnızca ihtiyaç duyan kuruma yükler.

Aktör / Kaynak	Okuma	Senaryo Tanımlama	Koşum Tetikleme	Çalışma Kopyası Üretimi	Temel Ağın Değiştirilmesi
Trafik Planlayıcısı (arayüz)	Evet	Evet	Evet	Hayır	**Hayır mimari olarak engellenmiştir**
Orkestrasyon (AS2)	Evet	Evet	Evet	Hayır-AS3'e devreder	Hayır (yalnızca kopya üretir)
Ağ Yönetimi (AS3)	Evet	Hayır	Hayır	Evet (yalnızca çalışma kopyası üzerinde)	Hayır-mimarinin değişmez kuralı
Simülasyon işçisi (AS5)	Evet (kendi iş dizini)	Hayır	-	Hayır	Hayır (salt-okunur bağlama)
Kurulum yöneticisi (bootstrap)	Evet	-	-	-	Evet (yalnızca ilk kurulumda)

Kaynak: Kendi Çalışmamız

3.5.3. Girdi Doğrulama ve Alt Süreç İzolasyonu

Kullanıcı girdisinin alt süreçlere ulaşması nedeniyle, aşağıdaki savunma katmanları tasarıma dâhil edilmiştir. Bu katmanlar, Analiz Raporu'nda tanımlanan R17 (Güvenlik - Girdi Doğrulama ve İzolasyon) gereksiniminin mimari karşılığıdır:

- **Kapalı katalog denetimi:** Müdahale tipi, altı tanımlı değerden biri olmak zorundadır; serbest metin kabul edilmez. Hedef eleman kimliği, temel ağda fiilen var olan bir kenar veya düğüme karşılık gelmelidir.
- **Şema ve aralık doğrulaması:** Gelen JSON, beklenen yapıya karşı doğrulanır; sayısal alanlar (şerit sayısı, hız sınırı, zaman aralığı) tanımlı aralıklarla sınırlanır.
- **Kabuk (Shell) kullanılmaması:** Alt süreçler, komut dizesi birleştirilerek değil, argüman listesi olarak çağrılır. Böylece kabuk meta-karakterlerinin yorumlanması yoluyla komut enjeksiyonu olanağı ortadan kalkar.
- **Dosya yolu hapsi:** Tüm girdi ve çıktı yolları standart hâle getirilir ve ilgili işin çalışma dizini dışına çıkan yollar reddedilir; kullanıcı girdisi hiçbir zaman dosya yolu üretmekte doğrudan kullanılmaz.
- **Kaynak sınırlaması:** Her alt süreç bir zaman aşımı ile çalıştırılır; süre aşıldığında süreç sonlandırılır ve iş başarısız olarak işaretlenir. Ayrıca konteyner düzeyinde CPU ve bellek üst sınırları tanımlanır (bkz. Bölüm 3.3.1 kaynak profilleri); bu sınırlar aşıldığında alt süreç işletim sistemi tarafından sonlandırılır ve aynı BAŞARISIZ durumuna geçiş uygulanır.

Konteyner düzeyinde ek bir izolasyon katmanı bulunur; simülasyon işçisi ayrı bir konteynerde, kök olmayan (non-root) bir kullanıcıyla çalışır; temel ağ dizinini salt-okunur olarak bağlar; ve koşum sırasında dış ağ erişimine ihtiyaç duymadığından ağ erişimi kısıtlanabilir. Bu sayede olası bir istismarın etkisi, o işe ait çalışma dizinine hapsedilir.

3.5.4. KVKK ve GDPR Uyumu

Sistem, tasarım gereği gizlilik ilkesine göre kurgulanmıştır. Kullanılan veri kaynakları OpenStreetMap [2] ve NYC Open Data [14], [15] gibi kamuya açık ve toplulaştırılmış setlerdir; taksi seyahat kayıtları yalnızca bölge bazlı OD dağılımlarının doğrulanması amacıyla toplulaştırılmış biçimde kullanılır ve bireysel kayıtlar sistemde saklanmaz. Sistem kullanıcı hesabı tutmadığından ve kişisel veri barındıran bir veritabanı bulunmadığından, KVKK ve GDPR kapsamında işlenen kişisel veri yoktur.

Bu ilke mimaride şu somut karşılıklara sahiptir: Kalıcı depoda kişisel veri barındıran hiçbir dizin tanımlanmamıştır (bkz. Bölüm 3.4.4); uygulama günlükleri yalnızca teknik olayları ve iş kimliklerini içerir, kullanıcıyı tanımlayan bir alan taşımaz ve sistemin dış ağ erişimi yalnızca kurulum aşamasındaki açık veri indirmeleriyle sınırlıdır. Kullanılan veri kaynaklarının lisans yükümlülükleri (OpenStreetMap için ODbL atıf koşulu, NYC Open Data setleri için CC0) sistem dokümantasyonunda ve arayüzün bilgi bölümünde belirtilir.

3.5.5. Şeffaflık ve Model Sınırlılığının Bildirimi

Güvenliğin bir boyutu da, sistemin ürettiği bilginin yanlış yorumlanmasının önlenmesidir. Model, mekânsal dağılımı homojen olmayan açık verilere dayandığından bölgesel temsil yanlılığı taşıyabilir. Bu nedenle sunum katmanında, sonuç ekranlarında kalıcı bir bilgilendirme unsuru bulunur: Çıktıların mutlak gerçeklik değil, model temelli tahminler olduğu ve altyapı kararlarında tek dayanak olarak kullanılmaması gerektiği açıkça belirtilir. Aynı bilgi, dışa aktarılan raporlara da eklenir. Buna ek olarak her koşumun girdi varsayımları ve müdahale geçmişi JSON meta-verisi olarak saklandığından, bir sonucun hangi koşullar altında üretildiği geriye dönük olarak denetlenebilir.

3.6. Küresel Yazılım Denetimi

3.6.1. Denetim Akışı Modeli

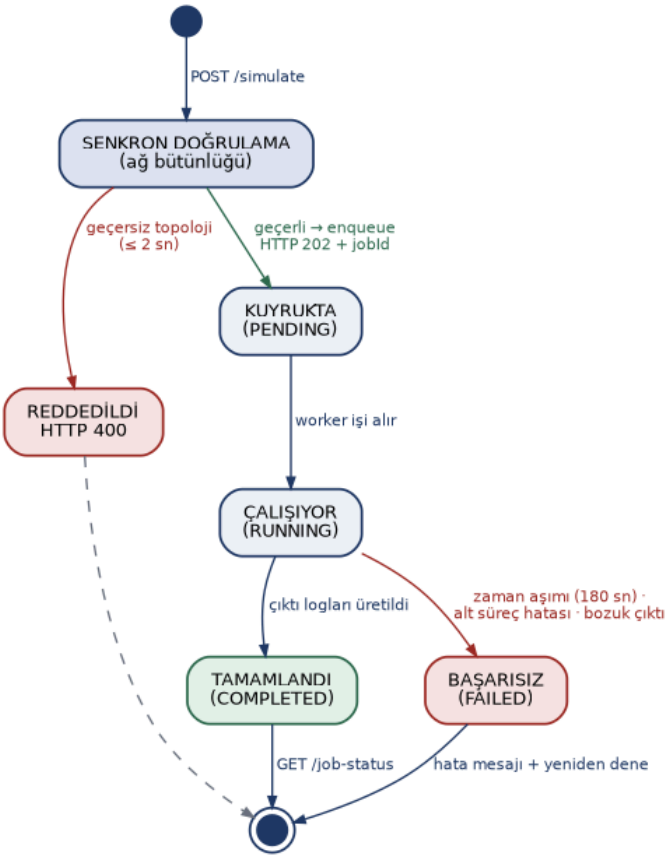
Sistemde üç farklı denetim biçimi bir arada kullanılır. Sunum katmanı **olay güdümlüdür**: kullanıcı etkileşimleri (harita tıklaması, katalog seçimi, koşum tetikleme) uygulamanın durumunu değiştirir. Orkestrasyon katmanı **istek-yanıt** modelinde çalışır. Simülasyon yürütme ise **kuyruk tabanlı asenkron** bir modelle yönetilir.

Bu ayrımın kritik noktası, hangi işlemin senkron, hangisinin asenkron olduğudur ve bu bilinçli bir tasarım kararıdır:

- **Senkron kalan:** ağ doğrulaması. Kullanıcı koşumu tetiklediğinde sistem işi hemen kuyruğa göndermez; önce çalışma kopyasını oluşturup doğrular. Bu adım hızlıdır (hedef: 2 saniyenin altında) ve senkron tutulmasının iki gerekçesi vardır. Birincisi, geçersiz bir ağ için kuyruk ve işçi kaynağı harcamak anlamsızdır. İkincisi, kullanıcı hatasının anında ve bağlamıyla birlikte bildirilmesi, dakikalar sonra "iş başarısız" mesajı almasından çok daha kullanışlıdır. Doğrulama başarısız olduğunda istek HTTP 400 ile reddedilir.
- **Asenkron olan:** simülasyon koşumu. Doğrulama geçildiğinde iş kuyruğa alınır ve istemciye derhal HTTP 202 (Accepted) yanıtı ile bir iş kimliği döndürülür. Arayüz bu noktadan sonra

bloklanmaz; kullanıcı harita üzerinde gezinmeye devam edebilirken bir ilerleme göstergesi işin durumunu yansıtır.

İstemci, iş durumunu düzenli aralıklarla (yaklaşık 3 saniye) sorgulayarak takip eder. Sunucu itmeli (WebSocket, sunucu gönderimli olaylar) bir çözüm yerine yoklamanın seçilmesi bir sadelik tercihidir: tek kullanıcılı ve dakikalar mertebesinde süren bir işlem için kalıcı bağlantı yönetimi gereksiz karmaşıklık üretir; ayrıca yoklama, bağlantı koptuğunda veya sayfa yenilendiğinde durumun kaybolmaması avantajını sağlar. Bunun bedeli, tamamlanma bilgisinin en fazla bir yoklama aralığı kadar gecikmesidir ve bu gecikme kabul edilebilir bulunmuştur.



Şekil 5. Simülasyon işinin yaşam döngüsü ve durum geçişleri

3.6.2. Eşzamanlılık Modeli

Sistem, aynı anda tek bir aktif senaryo ve tek bir çalışan iş varsayımıyla tasarlanmıştır. İşçi eşzamanlılığı bilinçli olarak bire sabitlenmiştir: simülasyon CPU'ya bağımlı bir yüktür ve referans donanımda (4 çekirdek) paralel iki koşum, her ikisinin de süresini uzatarak 180 saniyelik hedefi tehlikeye atardı. Kuyruk bu nedenle bir hız düzenleyici (load leveling) işlevi görür; birden fazla istek gelirse işler sıraya alınır ve teker teker yürütülür. Bu tercih, ölçeklenebilirliği sınırlamakla birlikte öngörülebilir koşum süresi sağlar ve dosya tabanlı kalıcılıkla uyumludur. Eşzamanlı yazma çakışması oluşmasını engeller.

3.6.3. Hata Kurtarma Stratejisi

Hata kurtarma tasarımının temel ilkesi, **geçici hatalar ile deterministik hataların ayrılmasıdır**. Geçici bir hata (broker bağlantısının anlık kopması gibi) yeniden denemeyle çözülebilir; deterministik

bir hata (geçersiz topoloji, bozuk girdi dosyası) ise aynı girdiyle her defasında aynı sonucu verecektir ve yeniden denemek yalnızca kaynak tüketir. Sistem bu ikisini ayırarak davranır.

Hata Sınıfı	Örnek	Tespit Yöntemi	Kurtarma Stratejisi
Deterministik girdi hatası	İzole düğüm üreten müdahale; geçersiz müdahale tanımı	Senkron doğrulama kapısı	**Yeniden deneme yok.** İstek HTTP 400 ile reddedilir; kullanıcıya hangi denetimin başarısız olduğu ve düzeltme yolu bildirilir.
Geçici altyapı hatası	Mesaj kuyruğuna erişilememesi; anlık dosya kilidi	Bağlantı/işlem istisnası	Üstel geri çekilmeli sınırlı yeniden deneme (en fazla iki kez); başarısızlık sürerse iş başarısız işaretlenir.
Alt süreç başarısızlığı	duarouter'ın rota kuramaması; netconvert hatası	Süreç çıkış kodu ve hata akışı	Yeniden deneme yapılmaz; alt sürecin hata çıktısı iş kaydına yazılır ve kullanıcıya anlaşılır bir özet gösterilir.
Zaman aşımı	Koşumun 180 saniyelik sınırı aşması	Alt süreç zamanlayıcısı	Süreç sonlandırılır (abort), iş BAŞARISIZ durumuna geçer, kısmi çıktılar temizlenir; kullanıcıya ölçeği küçültme önerisi sunulur.
Yakınsama sorunu	DUA iterasyonlarının kararlı dengeye ulaşmaması	İterasyonlar arası rota değişim oranı	**Zarif bozulma (fallback):** iterasyon sayısı üst sınıra kadar çalıştırılır ve "yeterince iyi dağılım" sonucu, yakınsama uyarısıyla birlikte sunulur.
Kaynak eşiği aşımı	fcd çıktısının kabul edilemez boyuta ulaşması; disk doluluğu	Dosya boyutu ve disk denetimi	**Zarif bozulma:** mikroskobik çıktı devre dışı bırakılır, yalnızca segment bazlı çıktı kullanılır; eski iş dizinleri otomatik temizlenir.
Ardışık sistemik hata	İşçinin arka arkaya birçok işi başarısız tamamlaması	Ardışık hata sayacı	**Devre kesici:** kuyruk geçici olarak duraklatılır ve arayüzde sistem durumu uyarısı gösterilir; böylece hatalı kurulum sessizce kaynak tüketmez.

Kaynak: Kendi Çalışmamız

Kurtarma tasarımını destekleyen iki mimari özellik vardır. **İdempotenlik:** her iş kendi izole çalışma dizininde çalıştığından, başarısız bir işin yeniden çalıştırılması yan etki üretmez; temel ağ ve talep artefaktları değişmediği için sonuç tekrar üretilebilir. **Korelasyon:** tüm günlük kayıtları iş kimliğiyle etiketlenir, böylece bir başarısızlık zinciri arayüzden alt süreç çıktısına kadar izlenebilir.

3.7. Sınır Koşulları

3.7.1. Soğuk Başlangıç (Cold Start)

Sistemin ilk kez ayağa kaldırılmasındaki iki aşama

1. **Birinci aşama:** Konteynerlerin başlatılmasıdır mesaj kuyruğu, ardından orkestrasyon sunucusu ve işçi, en son sunum bileşeni başlatılır. Bağımlılık sırasının korunması için sağlık

denetimleri (healthcheck) tanımlanır; orkestrasyon sunucusu kuyruk erişilebilir olmadan istek kabul etmeye başlamaz.

2. **İkinci aşama:** Veri hazırlığının denetimidir. Sistem açılışta temel ağın ve talep artefaktlarının var olup olmadığını kontrol eder. Bunlar yoksa arayüz doğrudan çalışma ekranına değil, kurulum akışına yönlendirir; çünkü temel ağ olmadan hiçbir senaryo tanımlanamaz. Bu, mimarinin "eksik veriyle yanlış çalışmaktansa açıkça durmak" ilkesinin bir uygulamasıdır.

Kullanıcı deneyimi açısından kritik bir tasarım kararı, **temel senaryo metriklerinin önceden hesaplanmasıdır**. Karşılaştırmalı analiz her zaman bir referans noktası gerektirdiğinden, temel senaryonun koşumu kurulum aşamasında bir kez yapılır ve sonuçları saklanır. Böylece kullanıcı arayüzü her açtığında temel senaryo haritası ve özet göstergeleri anında (3 saniyelik render hedefi içinde) görüntülenir; ilk açılışta dakikalarca beklemek gerekmez.

3.7.2. Kapatma (Graceful Shutdown)

Sistem kapatılırken veri tutarlılığını korumak amacıyla işlemler aşağıdaki sıraya göre gerçekleştirilir:

1. **Yeni İş Kabulünün Durdurulması:** İlk olarak orkestrasyon sunucusu dışarıdan gelen yeni istekleri ve iş kabulünü tamamen durdurur.
2. **Çalışan Süreçlerin Sonlandırılması:** İşçi (worker), o an çalışmakta olan alt sürece güvenli bir sonlandırma sinyali gönderir. Tanımlı bir bekleme süresi dolduğunda, süreç zorla kapatılır.
3. **İş Durumlarının Güncellenmesi:** Kapanma anında işlemde olan (yarım kalan) iş "BAŞARISIZ" olarak işaretlenerek gerekçesi günlüklere kaydedilir. Henüz çalışmaya başlamamış, kuyrukta bekleyen işler ie sistemin bir sonraki açılışı için yeniden kuyruğa alınır.
4. **Geçersiz Çıktıların Temizlenmesi:** Kapanış sırasında yarım yazılmış veri dosyaları (çıktılar), atomik yazma kuralı gereği henüz nihai adlarına taşınmamış olduklarından geçerli veri olarak görünmez ve sistemden temizlenerek çöp veri oluşumu engellenir.

3.7.3. Hata ve İstisna Durumları

Aşağıdaki tablo, sistemin sınır durumlarında nasıl davranacağını özetlemektedir. Tabloda yer alan tepkiler, kullanıcıya her durumda anlaşılır bir açıklama ve mümkünse bir düzeltme yolu sunma ilkesine dayanır.

Sınır Durumu	Sistem Tepkisi
Temel ağ dosyası bulunamıyor veya bozuk	Sistem çalışma ekranını açmaz; kurulum akışına yönlendirir ve eksik artefaktı bildirir.
Talep artefaktları eksik veya seçilen zaman aralığı için üretilmemiş	Koşum tetiklenmez; kullanıcıya talep hazırlama adımının çalıştırılması gerektiği bildirilir.
Müdahale sonucu izole düğüm oluşuyor	Doğrulama kapısı işi reddeder (HTTP 400); hangi bölgenin erişilemez kaldığı belirtilir ve "müdahaleyi geri al" seçeneği sunulur.
netconvert bağlantıları yeniden hesaplayamıyor	Çalışma kopyası geçersiz sayılır; iş kuyruğa alınmaz, alt sürecin hata özeti kullanıcıya iletilir.
duarouter bazı araçlar için rota kuramıyor	Koşum, rotalanamayan araçlar atlanarak sürdürülür; sonuç ekranında etkilenen talep oranı uyarı olarak gösterilir.

Sınır Durumu	Sistem Tepkisi
Koşum 180 saniyeyi aşıyor	İş sonlandırılır ve BAŞARISIZ işaretlenir; kullanıcıya zaman aralığını daraltma veya müdahale sayısını azaltma önerilir.
Çıktı XML'i eksik veya bozuk üretilmiş	Ayrıştırma hatası yakalanır; iş BAŞARISIZ olur ve arayüzde "metrikler yüklenemedi" bildirimi gösterilir.
Disk alanı eşiğin altına düşüyor	Yeni iş kabul edilmez; en eski iş çalışma dizinleri otomatik temizlenir ve durum arayüzde bildirilir.
Mesaj kuyruğuna veya işçiye erişilemiyor	Koşum isteği kabul edilmez; sınırlı yeniden denemeden sonra sistem durumu uyarısı gösterilir (devre kesici).
Kullanıcı sayfayı yeniliyor veya sekmeyi kapatıp geri dönüyor	İş sunucu tarafında sürdüğünden etkilenmez; arayüz iş kimliği üzerinden durumu yeniden sorgulayarak ilerleme ekranına geri döner.
Bir iş sürerken ikinci kez koşum tetikleniyor	İkinci istek kuyruğa alınır; arayüz aktif bir işin sürdüğünü bildirir ve tekrarlı gönderimi engeller.
Boş senaryo ile koşum tetikleniyor (hiç müdahale yok)	İstek geçerli sayılır ve temel senaryonun yeniden koşumu olarak yürütülür; kullanıcı bilgilendirilir.

Kaynak: Kendi Çalışmamız

4. ALT SİSTEM SERVİSLERİ

Bu bölüm, her alt sistemin dış dünyaya sunduğu servisleri, bu servisleri kimin tükettiğini ve hangi gereksinimi karşıladığını tek bir tabloda toplamaktadır. Servisler, davranışsal sözleşme düzeyinde tanımlanmıştır; parametre listeleri, dönüş tipleri ve hata kodlarının tam kümesi LLD dokümanında kesinleştirilecektir.

Alt Sistem	Sunulan Servis	Tüketici	Etkileşim	İlgili Gereksinim
AS1-Sunum	Harita üzerinde ağ görüntüleme ve eleman seçimi	Kullanıcı	Olay güdümlü	R1, R3
AS1-Sunum	Müdahale kataloğunun sunulması ve senaryo sepetinin yönetimi	Kullanıcı	Olay güdümlü	R3
AS1-Sunum	Karşılaştırmalı sonuç görselleştirmesi (ısı haritası, tablo, grafik) ve uyarı gösterimi	Kullanıcı	Olay güdümlü	R9, R10, R14
AS1-Sunum	Zaman aralığı seçim arayüzü ve varsayılan Peak Hour ayarı	Kullanıcı	Olay güdümlü	R7
AS2-Orkestrasyon	Senaryo yaşam döngüsü yönetimi (oluşturma, müdahale ekleme/çıkarma)	AS1	REST / JSON	R3

Alt Sistem	Sunulan Servis	Tüketici	Etkileşim	İlgili Gereksinim
AS2-Orkestrasyon	Koşum tetikleme ve iş kimliği döndürme	AS1	REST (HTTP 202)	R6
AS2-Orkestrasyon	İş durumu sorgulama ve sonuç teslimi	AS1	REST (yoklama)	R6, R8
AS2-Orkestrasyon	Simülasyon zaman aralığının seçimi ve koşum parametrelerine aktarımı	AS1	REST / JSON	R7
AS2-Orkestrasyon	Doğrulama hatası durumunda gerekçeli HTTP 400 yanıtı üretimi	AS1	REST (hata yanıtı)	R15
AS2-Orkestrasyon	Gelen istek JSON'unun şema ve aralık doğrulaması	AS1	Süreç içi çağrı	R17
AS3-Ağ Yönetimi	OSM verisinden temel ağ üretimi (kurulum)	Kurulum akışı	Alt süreç (netconvert)	R1
AS3-Ağ Yönetimi	Çalışma kopyası üretimi ve müdahalelerin uygulanması	AS2	Süreç içi çağrı	R3, R5
AS3-Ağ Yönetimi	Ağ bütünlüğü doğrulaması ve gerekçeli ret	AS2	Süreç içi çağrı	R4, R15
AS3-Ağ Yönetimi	Kapalı müdahale kataloğundaki altı tipin ayrı ayrı işlenmesi (şerit kapatma, tek yön, hız değişimi, kavşak dönüşümü, slip-lane, bypass)	AS2	Süreç içi çağrı	R18
AS4-Talep Modelleme	Ham veri temizleme ve saatlik hacim profili çıkarımı	AS2	Süreç içi çağrı	R2
AS4-Talep Modelleme	Çekim Modeli ile OD matrisi üretimi	AS2	Süreç içi çağrı	R2, R12
AS4-Talep Modelleme	OD matrisinin seyahat tanımlarına dönüştürülmesi	AS2	Alt süreç (od2trips)	R2
AS5-Simülasyon Yürütme	İşin kuyruğa alınması ve durum geçişlerinin yönetimi	AS2	Mesaj kuyruğu	R6
AS5-Simülasyon Yürütme	DUA iterasyonlu rota üretimi	İş kuyruğu	Alt süreç (duarouter)	R2

Alt Sistem	Sunulan Servis	Tüketici	Etkileşim	İlgili Gereksinim
AS5-Simülasyon Yürütme	Mikroskobik koşumun yürütülmesi ve zaman aşımı denetimi	İş kuyruğu	Alt süreç (sumo)	R6, R13
AS5-Simülasyon Yürütme	Alt süreç çağrılarının kabuksuz (argüman listesi) ve zaman aşımlı çalıştırılması	İş kuyruğu	Alt süreç izolasyonu	R17
AS6-Analiz ve Raporlama	Çıktı loglarının ayrıştırılması ve metrik hesaplama	AS2	Süreç içi çağrı	R8
AS6-Analiz ve Raporlama	Temel ve müdahale senaryolarının karşılaştırılması	AS2	Süreç içi çağrı	R9
AS6-Analiz ve Raporlama	Eşik tabanlı darboğaz uyarılarının üretilmesi	AS2	Süreç içi çağrı	R10
AS6-Analiz ve Raporlama	Doğrulama metriklerinin (GEH, KL) hesaplanması	Doğrulama akışı	Toplu çalıştırma	R11, R12
AS6-Analiz ve Raporlama	Metrik çıktısının toplulaştırılmış (istemci render sınırına uygun) JSON sözleşmesine dönüştürülmesi	AS2	Süreç içi çağrı	R14

Kaynak: Kendi Çalışmamız

5. HLD - LLD İZLENEBİLİRLİĞİ

Yüksek seviye tasarımın değeri, düşük seviye tasarıma izlenebilir biçimde açılabilmesiyle ölçülür. Bu bölüm, "bu HLD bloğu LLD'de nerede karşılık buluyor?" sorusuna cevap verecek eşlemeyi sunar. Eşlemede üç kural benimsenmiştir: (i) her gereksinim en az bir alt sisteme atanır, (ii) her HLD alt sistemi LLD'de bir veya daha fazla bileşen grubu olarak açılır, (iii) isimlendirme her iki dokümanda aynı kalır; LLD'de bir bileşen yeniden adlandırılacaksa HLD de güncellenir.

Gereksinim	Özet	HLD Alt Sistemi	LLD'de Açılacak Bileşen Grubu
R1	Sabit ağ topolojisinin yüklenmesi	AS3, AS1	Ağ üretim ve OSM dönüştürme bileşenleri; harita görüntüleme bileşenleri
R2	Talep üretimi (OD matrisi → rota dosyası)	AS4, AS5	Veri temizleme, Çekim Modeli hesaplama ve seyahat/rota dönüştürme bileşenleri
R3	Kontrollü müdahale uygulama	AS1, AS3	Müdahale kataloğu ve sepet bileşenleri; müdahale uygulama bileşenleri

Gereksinim	Özet	HLD Alt Sistemi	LLD'de Açılacak Bileşen Grubu
R4	Ağ doğrulama	AS3	Bağlantısallık ve izole düğüm denetim bileşenleri
R5	Temel ağ bütünlüğü	AS3	Çalışma kopyası yönetim bileşenleri (salt-okunur erişim politikası dâhil)
R6	Asenkron simülasyon yürütümü	AS2, AS5	İş gönderim ve durum yönetim bileşenleri; işçi görev bileşenleri
R7	Simülasyon zaman aralığı seçimi	AS1, AS2	Zaman seçici arayüz bileşeni; koşum parametresi eşleme bileşeni
R8	Metrik analizi	AS6	Log ayrıştırma ve metrik toplulaştırma bileşenleri
R9	Karşılaştırmalı görselleştirme	AS6, AS1	Karşılaştırma hesaplama bileşeni; ısı haritası ve grafik çizim bileşenleri
R10	Kural tabanlı uyarı	AS6, AS1	Eşik değerlendirme bileşeni; uyarı işaretleyici ve bilgi balonu bileşenleri
R11	Hacim tutarlılığı (GEH)	AS6	Doğrulama metriği hesaplama bileşenleri
R12	OD dağılım doğruluğu (KL)	AS4, AS6	Dağılım karşılaştırma ve metrik hesaplama bileşenleri
R13	Performans-koşum süresi	AS5	Zaman aşımı denetimi ve süreç yönetim bileşenleri
R14	Performans-arayüz render hızı	AS1, AS6	Toplulaştırılmış veri sözleşmesi; istemci çizim optimizasyon bileşenleri
R15	Hata mesajı yanıt süresi	AS2, AS3	Senkron doğrulama kapısı ve hata eşleme bileşenleri
R16	Taşınabilirlik	Dağıtım katmanı	Konteyner tanımları ve düzenleme dosyaları
R17	Girdi doğrulama ve izolasyon	AS2, AS5	Şema doğrulama, yol kanonikleştirme ve alt süreç çağrı bileşenleri
R18	Müdahale başarı oranı	AS3	Müdahale tipi işleyici bileşenleri (altı tip için ayrı ayrı)

Kaynak: Kendi Çalışmamız

Tasarım sürecinde kaçınılması gereken tipik hatalar da bu izlenebilirlik disiplininin gerekçesidir: HLD'de fonksiyon düzeyine inmek, LLD'de yeni mimari kararlar almak, HLD modüllerini LLD'de tamamen değiştirmek veya iki doküman arasında isim tutarsızlığı yaratmak. Yukarıdaki matris, LLD aşamasında bu sapmaların erken fark edilmesini sağlayacak referans noktasıdır.

6. SÖZLÜK

Alt süreç (subprocess). Bir programın, işletim sistemi düzeyinde ayrı bir süreç olarak başka bir programı çalıştırması. Bu projede SUMO araçları bu yolla çağrılır; iletişim yalnızca dosyalar ve süreç çıkış kodları üzerinden kurulur.

Artefakt. Sistem tarafından üretilen ve dosya sisteminde saklanan somut çıktı: ağ dosyası, rota dosyası, iş kaydı, ham log veya hesaplanmış metrik dosyası.

Atomik yazma. Bir dosyanın önce geçici adla yazılıp yalnızca tamamlandığında nihai adına taşınması. Yarım kalmış bir çıktının geçerli veri olarak okunmasını önler.

Çekim Modeli (Gravity Model). İki bölge arasındaki seyahat talebini, bölgelerin çekicilikleri (nüfus, iş yoğunluğu) ile aralarındaki mesafenin bir fonksiyonu olarak tahmin eden klasik ulaştırma planlaması yöntemi [9].

Devre kesici (circuit breaker). Ardışık hatalar belirli bir sayıyı aştığında ilgili işlevi geçici olarak devre dışı bırakan hata izolasyon örüntüsü. Hatalı bir kurulumun sessizce kaynak tüketmesini engeller.

DUA (Dinamik Kullanıcı Dengesi). Wardrop'un birinci prensibine [3] dayanan iteratif rota atama yöntemi. Araçlar önce en kısa rotaya atanır; oluşan tıkanıklıklar hafızaya alınarak simülasyon yinelenir ve hiçbir sürücünün rota değiştirerek süresini kısaltamayacağı denge noktasına yaklaşılır.

Edge / Node (Kenar / Düğüm). Çizge kuramı terimleri. Kenar, yönlü bir yol segmentini (şerit sayısı, uzunluk, hız sınırı gibi fiziksel nitelikleri taşır); düğüm ise kavşak, çıkmaz sokak ucu veya yol birleşim noktasını temsil eder.

GEH İstatistiği. Gözlemlenen gerçek trafik sayımları ile simülasyonun ürettiği hacim tahminleri arasındaki farkı ölçen ampirik istatistik [4]. Düşük hacimli yollarda küçük sapmaların yarattığı yüksek yüzdesel hataları tolere edecek biçimde tasarlanmıştır; 10,0'ın altındaki değerler kabul edilebilir kalibrasyon eşiği sayılır.

Idempotenlik. Bir işlemin birden çok kez çalıştırılmasının, bir kez çalıştırılmasıyla aynı sonucu vermesi. Bu projede her işin izole dizinde çalışması, başarısız işlerin güvenle yeniden denenmesini sağlar.

İzole düğüm. Bir müdahale sonucunda ağın geri kalanıyla bağlantısı kesilen düğüm veya düğüm grubu. Rota algoritmalarının çözüm bulamamasına yol açtığından, simülasyon öncesinde tespit edilerek engellenir.

KL Iraksama (Kullback-Leibler). Bir olasılık dağılımının bir referans dağılımdan ne kadar farklı olduğunu ölçen asimetrik metrik [5]. Projede sentetik OD dağılımının gerçek seyahat örüntülerine yakınlığını kanıtlamak için kullanılır.

Modüler monolit. Tek bir dağıtım birimi olarak paketlenen, ancak iç sorumlulukları net sınırlarla ayrılmış mimari yaklaşım. Mikroservislerin operasyonel maliyetine katlanmadan modülerlik sağlar.

OD Matrisi (Başlangıç-Varış). Belirli bir zaman diliminde hangi bölgeden hangi bölgeye ne kadar seyahat talebi olduğunu gösteren kare matris. Satırlar çıkış, sütunlar varış bölgelerini temsil eder.

Ters vekil sunucu (reverse proxy). İstemci ile uygulama arasında konumlanan ve TLS sonlandırma, kimlik doğrulama veya yönlendirme gibi işlevleri uygulama kodunu değiştirmeden sağlayan ara sunucu.

Yoklama (polling). İstemcinin sunucudaki durumu belirli aralıklarla sorgulaması. Kalıcı bağlantı gerektiren itmeli yöntemlere göre daha basittir ve bağlantı kopmalarına karşı dayanıklıdır.

Zarif bozulma (graceful degradation). Bir bileşen tam kapasiteyle çalışamadığında sistemin tamamen durmak yerine daha sınırlı ama kullanışlı bir çıktıyla hizmet vermeye devam etmesi. Örnek: mikroskobik çıktı devre dışı bırakıldığında yalnızca segment bazlı ısı haritasının sunulması.

netconvert / od2trips / duarouter / sumo. SUMO araç setinin sırasıyla ağ dönüştürme ve doğrulama, OD matrisinden seyahat üretme, rota hesaplama ve mikroskobik koşum yapan komut satırı programları.

edgedata-output.xml / fcd-output.xml. SUMO'nun ürettiği iki çıktı biçimi. Birincisi yol segmenti düzeyinde toplulaştırılmış (makroskobik) özet verir ve ısı haritalarının kaynağıdır; ikincisi her aracın her saniyedeki konumunu içeren ayrıntılı (mikroskobik) kayıttır.

7. KAYNAKLAR

[1] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC), Maui, HI, USA, 2018, pp. 2575-2582. doi: 10.1109/ITSC.2018.8569938

[2] OpenStreetMap contributors, "OpenStreetMap," 2025. [Çevrimiçi]. Erişim: https://www.openstreetmap.org, Erişim Tarihi: 05.08.2026

[3] J. G. Wardrop, "Some theoretical aspects of road traffic research," Proc. Inst. Civ. Eng., vol. 1, no. 3, pp. 325-362, 1952. doi: 10.1680/ipeds.1952.11259

[4] Federal Highway Administration, "Traffic Analysis Toolbox Volume III: Guidelines for Applying Traffic Microsimulation Modeling Software," U.S. Dept. Transp., Washington, DC, USA, Rep. FHWA-HRT-04-040, 2004.

[5] S. Kullback ve R. A. Leibler, "On information and sufficiency," Ann. Math. Stat., vol. 22, no. 1, pp. 79-86, 1951. doi: 10.1214/aoms/1177729694

[6] PTV Group, "PTV Vissim User Manual," Karlsruhe, Germany, t.y. [Çevrimiçi]. Erişim: https://www.ptvgroup.com, Erişim Tarihi: 05.08.2026

[7] A. Horni, K. Nagel, ve K. W. Axhausen, Eds., The Multi-Agent Transport Simulation MATSim. London, U.K.: Ubiquity Press, 2016.

[8] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," Computers, Environment and Urban Systems, vol. 65, pp. 126-139, 2017.

[9] A. G. Wilson, "A statistical theory of spatial distribution models," Transp. Res., vol. 1, no. 3, pp. 253-269, 1967. doi: 10.1016/0041-1647(67)90035-4

[10] S. Ramírez, "FastAPI," 2018. [Çevrimiçi]. Erişim: https://fastapi.tiangolo.com/, Erişim Tarihi: 05.08.2026

[11] Meta Platforms, Inc., "React," 2013. [Çevrimiçi]. Erişim: https://react.dev/, Erişim Tarihi: 05.08.2026

[12] V. Agafonkin, "Leaflet: an open-source JavaScript library for mobile-friendly interactive maps," t.y. [Çevrimiçi]. Erişim: https://leafletjs.com/, Erişim Tarihi: 05.08.2026

[13] Eclipse Foundation, "Eclipse Public License v2.0," 2024. [Çevrimiçi]. Erişim: https://www.eclipse.org/legal/epl-2.0/, Erişim Tarihi: 05.08.2026

[14] NYC Open Data, "Automated Traffic Volume Counts," New York City Department of Transportation, t.y. [Çevrimiçi]. Erişim: https://data.cityofnewyork.us, Erişim Tarihi: 05.08.2026

[15] NYC Department of City Planning, "Primary Land Use Tax Lot Output (PLUTO) and MapPLUTO," New York City Open Data, t.y. [Çevrimiçi]. Erişim: https://www.nyc.gov/content/planning/pages/resources/datasets/mappluto-pluto-change, Erişim Tarihi: 05.08.2026

8. ARAÇLAR VE METODOLOJİ

Bu dokümanın hazırlanma sürecinde (fikir geliştirme, metin/tablo düzenleme, mimari diyagram görselleri ve bölümlerin ilk taslakları) ChatGPT, Claude ve Gemini araçlarından destek alınmıştır. Ancak tüm teknik kararlar, sistem mimarisi ve nihai kontroller proje ekibi tarafından yapılmış olup, içeriğin doğruluğu, özgünlüğü ve teknik yeterliliği konusundaki tüm sorumluluk ekibimize aittir.